

Semesterprojekt

Applied AI · SS26 · Prof. Dr. Pascal Laube · HTWG Konstanz

Überblick

Statt einer Klausur schließt ihr diesen Kurs mit einem **eigenen Applied-AI-Projekt** ab. Ihr entwickelt ein System, das ihr selbst gewählt habt – die einzige Vorgabe ist, dass es sich inhaltlich auf den Bereich Applied AI bezieht und die Konzepte aus dem Kurs erkennbar umsetzt.

■ Was ist das Ziel?

Ihr baut ein lauffähiges System im Bereich **Applied AI**, das mindestens einen echten AI Agent enthält. Ihr wählt Domäne, Technologie-Stack und Komplexität selbst.

Zur Bewertung gilt: **Alle Pflichtenforderungen (P1–P5) müssen erfüllt sein**. Zusätzlich müssen mindestens **11 von 14 Wahlpflichtenforderungen** erfüllt sein – das entspricht ca. 80 %. Beides zusammen ist die Mindestvoraussetzung für eine bestandene Abgabe.

★ Kernaussage

Die Anforderungen sind kein Korsett – sie sind ein Kompass. Sie stellen sicher, dass euer Projekt die zentralen Themen des Kurses widerspiegelt. Innerhalb dieser Leitplanken habt ihr volle kreative Freiheit.

Teamgröße

Projekte werden in **2er-Teams** durchgeführt. Einzelprojekte sind nicht zugelassen, größere Gruppen ebenfalls nicht.

Projektidee einreichen (Pitch – 26.03. im Praktikum)

Am **26.03. im Praktikum** pitcht ihr eure Projektidee in wenigen Sätzen. Kein ausgearbeitetes Konzept nötig – es geht darum, die Grundidee zu kommunizieren und sicherzustellen, dass sie in den Rahmen des Kurses passt.

Der Pitch enthält folgende Punkte:

- **Projektname** und Kurzbeschreibung (2–3 Sätze): Was soll das System können?
- **Domäne / Anwendungsfall**: In welchem Kontext wird es eingesetzt?
- **Teammitglieder**: Beide Namen bekannt geben

Agent-Architektur und Framework müssen zu diesem Zeitpunkt **noch nicht feststehen** – das ergibt sich im Laufe des Projekts.

Die Projektidee muss von mir kurz bestätigt werden. Das schützt euch davor, in eine Sackgasse zu laufen.

Fortschritts-Check alle 3 Wochen

Im Praktikum präsentiert ihr alle drei Wochen euren aktuellen Stand – **maximal 5 Minuten**, keine Folien nötig, kurze Live-Demo oder Terminal-Output genügt. Der Check dient dazu, rechtzeitig

Probleme zu erkennen und Feedback zu geben. Er fließt **nicht in die Bewertung** ein, ist aber verpflichtend.

 Achtung

Wer beim Fortschritts-Check nichts vorweisen kann, riskiert eine Nachbesserungspflicht. Fangt früh an.

Abgabe

Zum Semesterende gebt ihr ein **Git-Repository** ab (GitHub oder GitLab, öffentlich oder mit Leserechten für mich). Das Repository enthält:

1. **Lauffähiger Code** – das System muss reproduzierbar startbar sein (ich muss es lokal zum Laufen bringen können)
2. **Ausführliche Projektdokumentation** als Markdown-Datei im Repository – nicht nur eine README, sondern eine vollständige Beschreibung des Systems: Architektur, Designentscheidungen, Funktionsweise, Grenzen
3. **Anforderungserfüllung** – eine klare, strukturierte Auflistung aller erfüllten Anforderungen (P1–P5 und gewählte W-Anforderungen) mit kurzem Nachweis pro Anforderung (Verweis auf Code, Screenshot, Log-Output oder Text)
4. **Reflexionstexte** für gewählte Reflexions-Anforderungen (W12–W14) ebenfalls als Markdown im Repository

Eine Live-Demo ist **nicht** Teil der Abgabe.

Anforderungskatalog

Pflichtanforderungen (P)

Alle fünf Pflichtanforderungen müssen erfüllt sein.

■ P1 – Echter AI Agent mit Tool-Use

Das System enthält mindestens einen AI Agent, der **echte Tools aufruft** (Web-Search, Code-Execution, Datei-Lesen, API-Call o. ä.). Ein reiner LLM-Direktaufruf ohne Tool-Use zählt nicht.

Nachweis: Log oder Screenshot, der mindestens einen Tool-Call zeigt.

■ P2 – TAO-Zyklus sichtbar

Bei einer repräsentativen Eingabe durchläuft der Agent mindestens **3 vollständige TAO-Schritte** (Thought → Action → Observation). Der Zyklus muss im Output nachvollziehbar sein (Verbose-Logging, Trace oder Print-Ausgabe).

Nachweis: Terminal-Output oder Screenshot mit mindestens 3 Iterationen.

■ P3 – Framework-Einsatz

Das System nutzt ein etabliertes Agent-Framework: **smolagents, LangGraph, LlamaIndex Agents, CrewAI** oder ein vergleichbares. Reiner Rohcode ohne Framework ist nicht zulässig.

Nachweis: Import-Statement + kurze Begründung im README, warum dieses Framework gewählt wurde.

■ P4 – Dokumentation (README)

Das Repository enthält eine **README-Datei** mit:

- Kurzbeschreibung des Systems (was tut es, für wen?)
- Architektur-Überblick (Textbeschreibung oder Diagramm)
- Installationsanleitung (Abhängigkeiten, .env-Variablen, Startbefehl)
- Kurzes Beispiel: Welche Eingabe führt zu welchem Verhalten?

■ P5 – Git-Repository mit sinnvoller Commit-Historie

Der Code liegt in einem Git-Repository. Die Commit-Historie zeigt **inkrementellen Entwicklungsfortschritt** – nicht einen einzelnen “Initial Commit” mit dem gesamten Projekt.

Nachweis: `git log --oneline` mit mindestens 10 Commits über den Projektzeitraum.

Wahlpflichtanforderungen (W)

Mindestens 11 der folgenden 14 Wahlpflichtanforderungen müssen erfüllt sein.

ID	Anforderung	VL-Bezug
W1	Multi-Agent-Setup: mind. 2 Agenten mit definierter Rollenaufteilung (Orchestrator + Subagent)	VL 4
W2	Multimodale Eingabe: mind. ein Agent verarbeitet eine nicht-textuelle Modalität als Input – z. B. Bilder, Audio, Video oder strukturierte Dateien (PDF, CSV)	VL 4
W3	RAG-Komponente: das System beantwortet Fragen über eine eigene Wissensbasis / eigene Dokumente	VL 7
W4	Agentic RAG: der Retrieval-Schritt wird vom Agenten selbst als Tool gesteuert, nicht hartcodiert	VL 7
W5	Observability: mind. eine Form von Tracing oder strukturiertem Logging ist eingebaut (z. B. Langfuse, Phoenix, strukturierte stdout-Ausgabe)	VL 7
W6	Prediction Service: der Agent / das Modell ist als HTTP-API erreichbar (z. B. FastAPI, Flask)	VL 9
W7	Containerisierung: das System startet vollständig via docker compose up (Dockerfile vorhanden)	VL 9/13
W8	Automatisiertes Testing: mind. 5 sinnvolle Unit- oder Integrationstests	VL 10
W9	Input-Validierung & Fehlerbehandlung: fehlerhafte oder unerwartete Eingaben werden graceful abgefangen	VL 9/10
W10	CI/CD-Pipeline: mind. ein automatischer Schritt bei Push (GitHub Actions, GitLab CI o. ä.)	VL 13
W11	Monitoring-Endpoint: /health-Route oder Prometheus-Metriken vorhanden	VL 11/13
W12	Data/Concept Drift – Reflexion (min. 1/2 Seite): Wie könnte Drift das System beeinflussen? Was würde auffallen?	VL 11
W13	Continual Learning – Konzept (min. 1/2 Seite): Wie könnte das System mit neuen Daten verbessert werden?	VL 12
W14	Responsible AI – Reflexion (min. 1/2 Seite): Welche Risiken, Biases oder Missbrauchspotenziale hat euer System?	VL 14

★ Kernaussage

W12, W13 und W14 sind **keine Programmieraufgaben** – das sind Reflexionstexte. Ihr könnt sie auch dann erfüllen, wenn euer System technisch einfach ist. Schreibt ehrlich und konkret über euer eigenes Projekt.

Projektideen zur Inspiration

- **Beispiel**
- **Recherche-Agent:** Ein Agent, der zu einem Thema selbstständig im Web recherchiert, Quellen bewertet und eine strukturierte Zusammenfassung erstellt.
- **Code-Review-Agent:** Ein Agent, der Pull Requests analysiert, typische Fehler erkennt und Verbesserungsvorschläge macht.
- **Lernassistent mit RAG:** Ein Chatbot, der Fragen über eigene Lernmaterialien (PDFs, Notizen) beantwortet – mit agentic Retrieval.
- **Daten-Analyse-Agent:** Ein Agent, der CSV-Dateien einliest, selbst Python-Code zur Analyse generiert und ausführt, und Ergebnisse erklärt.
- **Multimodaler Assistent:** Ein Agent, der eine nicht-textuelle Modalität verarbeitet – z. B. Bilder (Produktfotos analysieren), Audio (Sprachaufnahmen transkribieren und beantworten) oder PDFs (Dokumente einlesen und Fragen dazu beantworten).
- **Game-Playing Agent:** Ein Agent, der ein einfaches textbasiertes Spiel oder eine Simulation autonom löst.

Achtung

Was nicht zählt: Ein einfacher Chatbot ohne Tool-Use, ein reiner LLM-Wrapper (z. B. “ich rufe `openai.chat.completions.create` auf”), oder ein Demo-Skript aus einem Tutorial, das nur minimal angepasst wurde. Das Projekt muss **euer eigenes Werk** sein.